Capstone Project Report

Pizza Store Using Mern Stack

Under the Guidance of

Abhishek Sharma

Submitted By

Asa Bhargava
Lakshmi Narasimha
March-2026

1. Abstract

The **Pizza store system** is a full-stack web application created to simplify the online pizza ordering process. The platform enables customers to explore available pizza items, create an account, log in securely, add products to their shopping cart, and place orders conveniently. An administrative dashboard is also included to manage menu categories, products, users, and customer orders. The user interface is developed using **React**, which allows the application to provide a dynamic and responsive experience. Server-side operations are handled using **Node.js** with **Express.js**, which process client requests and implement the business logic of the system. The application stores data in **MongoDB**, where information related to users, products, and orders is maintained. Secure login functionality is implemented using **JSON Web Token** authentication. Overall, the project highlights the integration of frontend and backend technologies to build a modern and scalable web application.

2. Introduction

In recent years, online food ordering platforms have become increasingly popular because they allow customers to order food quickly and conveniently. Many restaurants are shifting towards digital systems that enable users to view menus, place orders, and manage purchases through online platforms. The **Pizza store** web application is designed to provide an efficient and easy-to-use system for ordering pizzas online. Customers can browse the available menu items, register an account, log into the system, and place orders using a simple interface. The application is developed using modern web technologies such as **React** for the frontend interface, while the backend is powered by **Node.js** and **Express.js**. All application data is stored in **MongoDB**, which enables efficient storage and retrieval of information. Additional functionalities such as authentication, cart operations, responsive design, and administrative controls are integrated into the system. The main objective of this project is to demonstrate the development of a modern full-stack application that is scalable, interactive, and user-friendly.

3. Objectives

- To design and implement a complete web application for ordering pizzas online.
- To allow customers to easily view available menu items and pizza products.
- To provide a secure login and registration system for users.
- To enable customers to add products to a cart and place orders efficiently.
- To create an intuitive and responsive interface that improves user interaction.

4. Technology Stack

- **Frontend:** React, Redux, Tailwind CSS
- **Backend:** Node.js, Express.js, Socket.IO
- **Database:** MongoDB
- **Authentication:** JSON Web Token
- **Testing:** Mocha, Chai, Super test, E2Etesting

5. System Architecture

The **Pizza store** platform follows a **three-layer architecture**, consisting of:

1. Presentation Layer (Frontend)

The frontend interface is built using **React**, which provides users with an interactive interface for viewing menu items, managing the cart, and completing the order process.

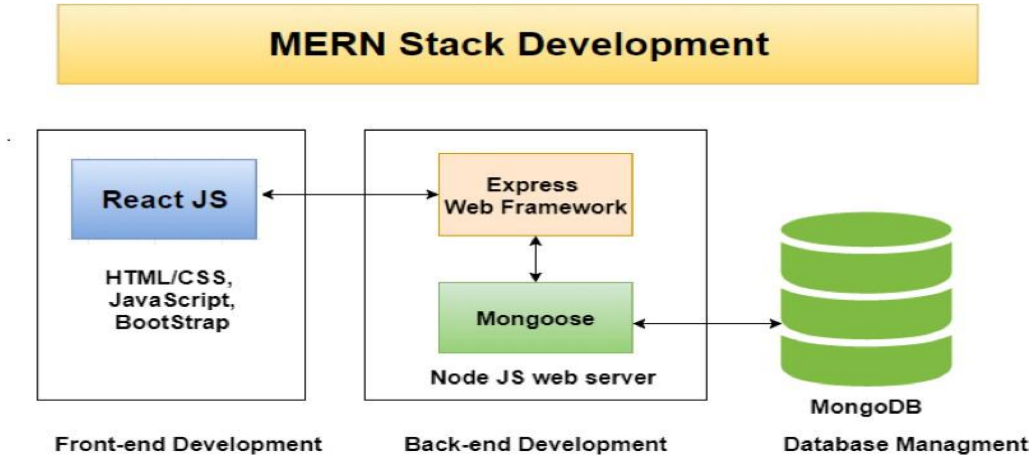
2. Application Layer (Backend)

The backend server is developed using **Node.js** and **Express.js**. This layer handles API requests, performs authentication, manages business logic, and communicates with the database.

3. Data Layer (Database)

The database layer uses **MongoDB** to store application data such as user profiles, menu items, categories, and order details.

Communication between the frontend and backend takes place through REST APIs, while **Mongoose** is used as an object modelling tool to interact with MongoDB. This architecture allows the application to remain scalable, organized, and efficient.



System Architecture of Pizza store.

6. Features Implemented:

- **Home Page:** Displays pizza products and categories available in the store
- **Authentication System:** Allows users to register and log in securely
- **Product Viewing:** Users can see pizza details including price, images, and descriptions
- **Cart Functionality:** Enables users to add and manage pizza items before ordering
- **Order Processing:** Customers can place orders and review order details
- **Admin Dashboard:** Administrators can manage products, categories, users, and orders
- **Notification System:** Alerts administrators about important activities such as new orders

7. Client-Side Validation

Client-side validation is implemented to verify user input before it is submitted to the server. This process helps ensure that users enter accurate and complete information in forms such as login, registration, and order details.

The system checks for conditions such as valid email formats, required input fields, and minimum password length. If incorrect data is entered, error messages are displayed immediately to guide the user. This approach improves user experience while also reducing unnecessary requests sent to the server.

8. Routing

Routing plays a significant role in managing navigation within the Pizza Hub application. On the frontend, **React Router** is used to switch between different pages such as the home page, login page, registration page, shopping cart, and admin panel without refreshing the browser. On the backend, routing is implemented using **Express.js**, which defines various API endpoints responsible for handling operations related to users, menu items, authentication, and orders. These routes allow the frontend to interact with the backend through structured REST API calls.

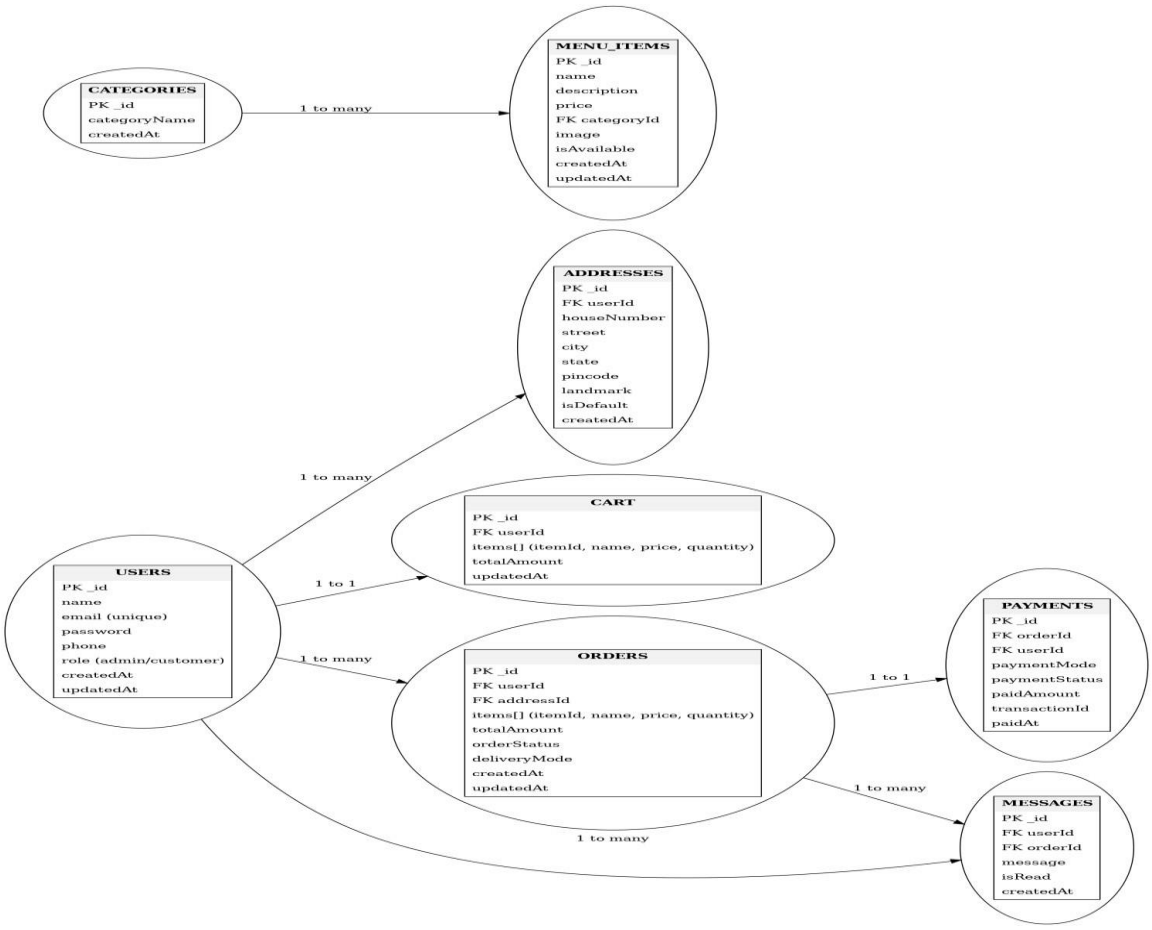
9.Frontend-Backend Integration

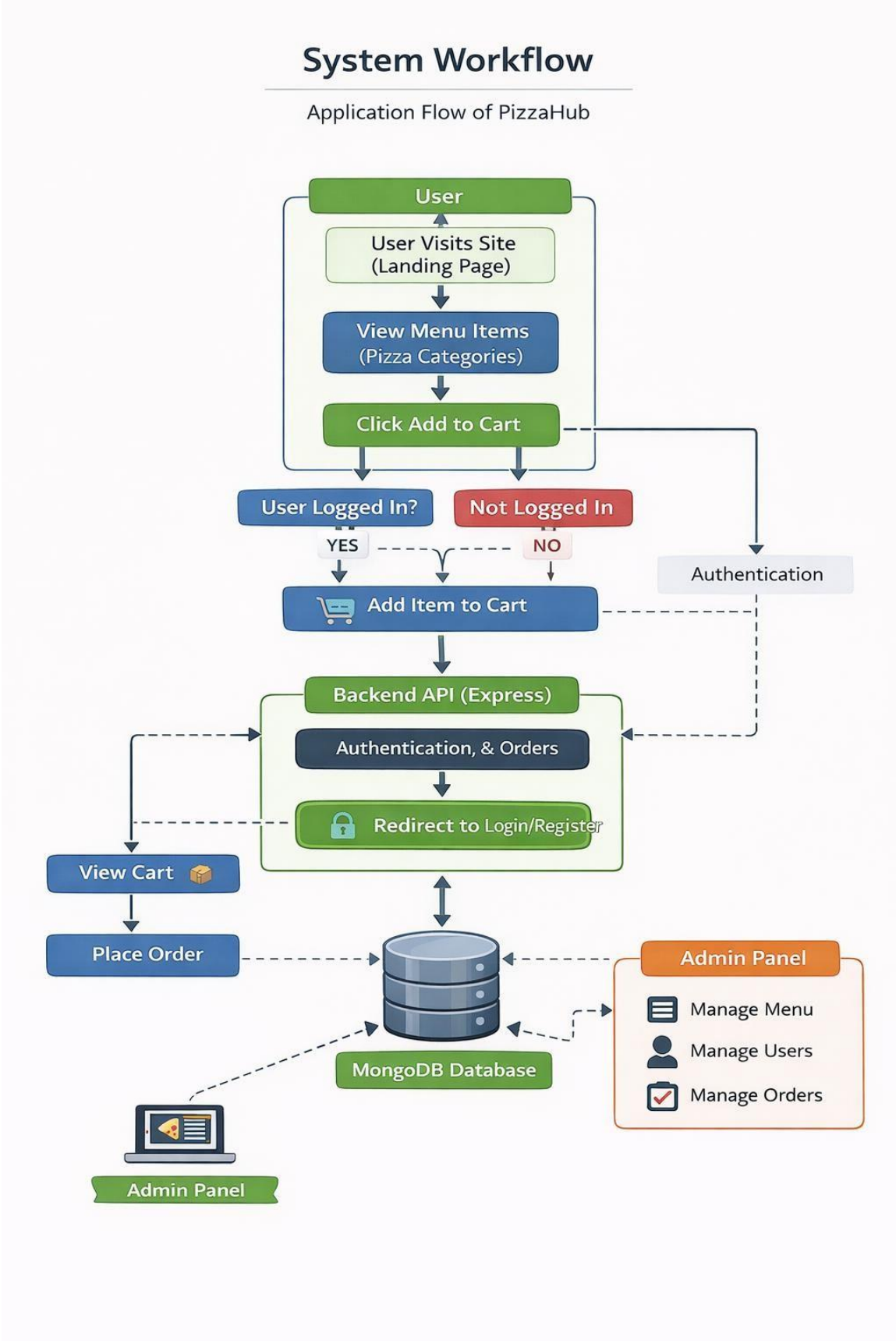
Integration between the frontend and backend components is achieved through RESTful APIs. The React interface sends HTTP requests to the Node.js server whenever a user performs an action such as logging in, retrieving menu items, adding products to the cart, or placing an order. These requests are typically handled using tools such as **Axios** or the Fetch API. The backend processes each request, communicates with the MongoDB database through **Mongoose**, and returns the appropriate response in JSON format. This ensures seamless communication between all components of the system

10.Testing

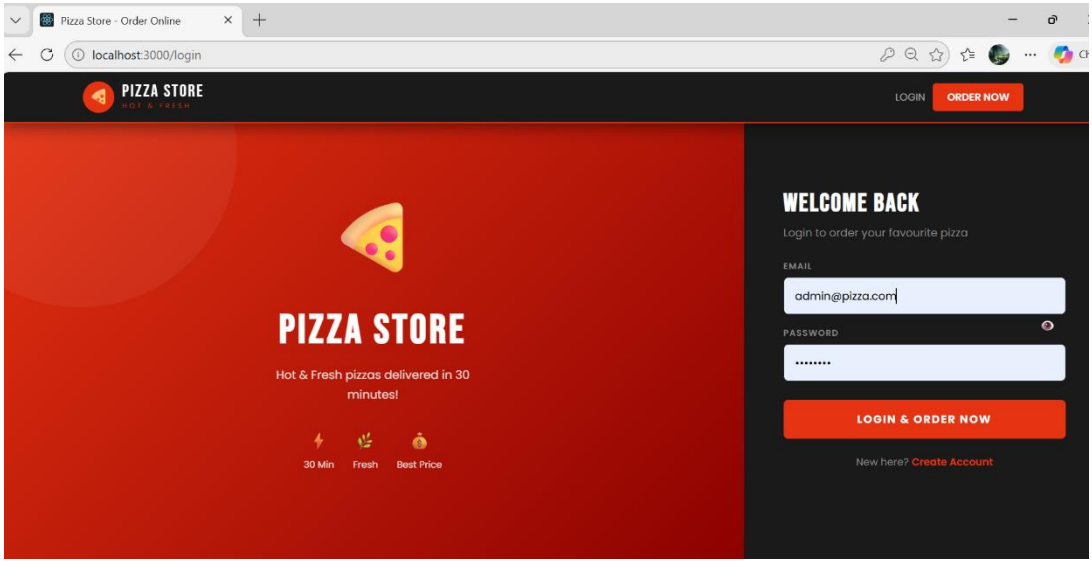
Testing is carried out to ensure that the application performs correctly and reliably. Different testing methods are applied during development. Unit testing is used to verify that individual functions and components behave as expected. Backend API endpoints are tested using tools such as **Mocha**, **Chai**, and **Super Test** to confirm that they return correct responses and handle errors properly. Additionally, End-to-End testing is performed to simulate real user actions such as signing in, adding items to the cart, and completing an order. These tests verify that all components of the system—from the user interface to the database—work together effectively.

11. Er Diagram And Application Workflow

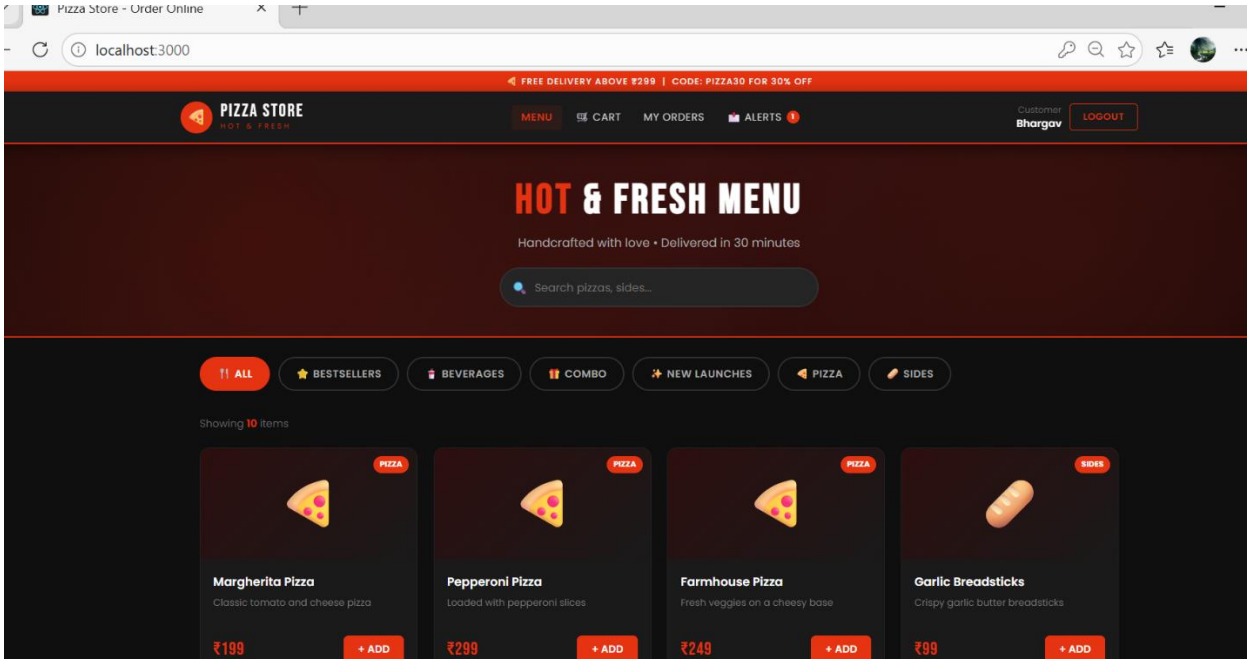



System Workflow
Application Flow of PizzaHub
User
User Visits Site (Landing Page)
View Menu Items (Pizza Categories)
Click Add to Cart
User Logged In?
Not Logged In
YES
NO
Authentication
Add Item to Cart
Backend API (Express)
Authentication, & Orders
Redirect to Login/Register
View Cart
Place Order
Admin Panel
Manage Menu
Manage Users
Manage Orders
MongoDB Database
Admin Panel

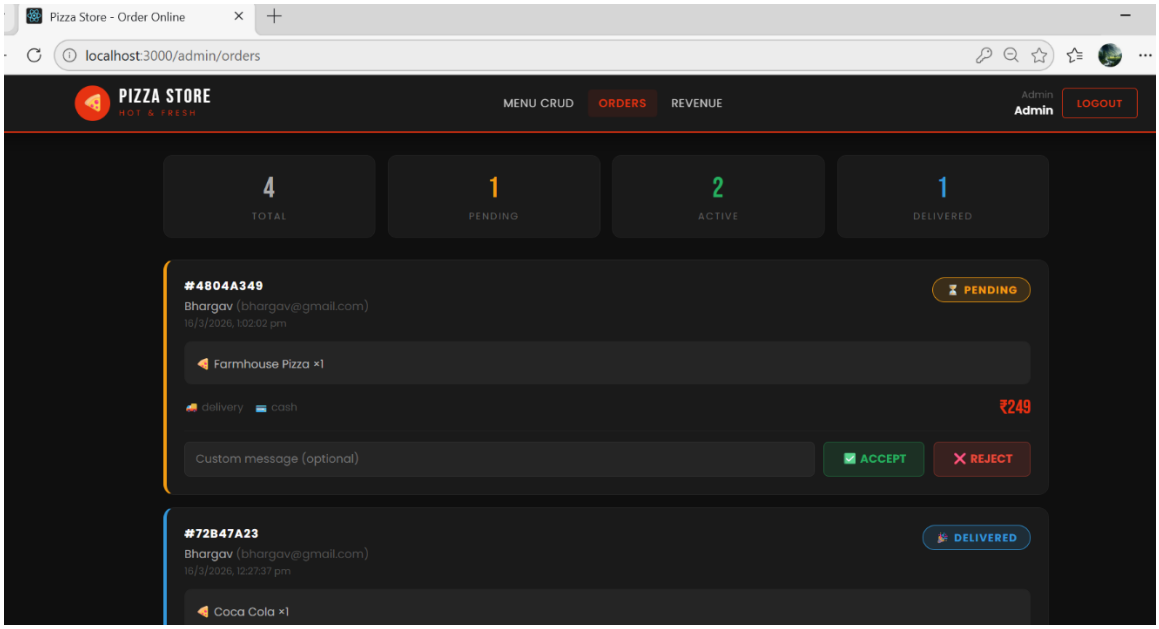
12. Application Screenshots:



Home page



User view



Admin View

13. Conclusion

The **Pizza Store** application demonstrates the successful development of a modern full-stack food ordering platform. Users can easily browse menu items, create accounts, add products to their carts, and place orders through a simple and responsive interface. The system integrates a **React-based frontend** with a **Node.js and Express backend**, while **MongoDB** provides efficient data storage. Important functionalities such as authentication, routing, input validation, and responsive design contribute to a smooth user experience. Furthermore , the inclusion of an admin panel allows efficient management of products, users, and customer orders. Testing methods were applied to improve system reliability and performance. Overall, the project highlights practical experience in building scalable web applications using modern development technology.